

COMP9517

Computer Vision

2026 Term 1 Week 2

A/Professor Yang Song



Image Processing

Part 2

Types of image processing (recap)

- Two main types of image processing operations:
 - **Spatial domain operations** (in image space)
 - **Transform domain operations** (mainly in Fourier space)
- Two main types of spatial domain operations:
 - **Point operations** (intensity transformations on individual pixels)
 - **Neighbourhood operations** (spatial filtering on groups of pixels)

Today

Topics and learning goals

- Describe the principles of the **Fourier transform for image processing**
Forward & inverse transform, convolution theorem, properties, discrete Fourier transform
- Understand the effects of various **Fourier domain filtering methods**
Filtering procedure, notch filtering, low-pass filtering, high-pass filtering
- Combine filtering operations to allow **multiresolution image processing**
Difference of Gaussians, image pyramids, approximation, reconstruction

What is lost when lowering resolution?



Downsampling



Upsampling



Jean Baptiste Joseph Fourier (1768-1830)

Had a crazy idea (1807)

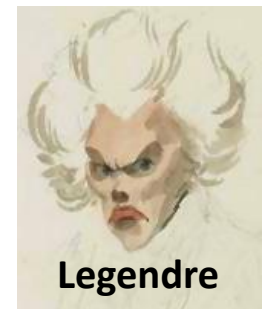
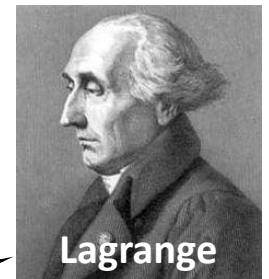
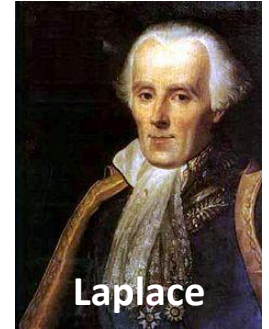
Any univariate function can be rewritten as a weighted sum of sines and cosines of different frequencies

Don't believe it?

- Neither did Lagrange, Laplace, Legendre, and other big wigs
- Fourier's idea was not translated into English until 1878

But it's (mostly) true!

- It is now called the Fourier series
- There are some subtle restrictions



"...the manner in which the author arrives at these equations is not exempt of difficulties and...his analysis to integrate them still leaves something to be desired on the score of generality and even rigour."

Weighted sum of sines

- **Basic building block**

$$f_i(x) = a_i \sin(\omega_i x + \varphi_i)$$

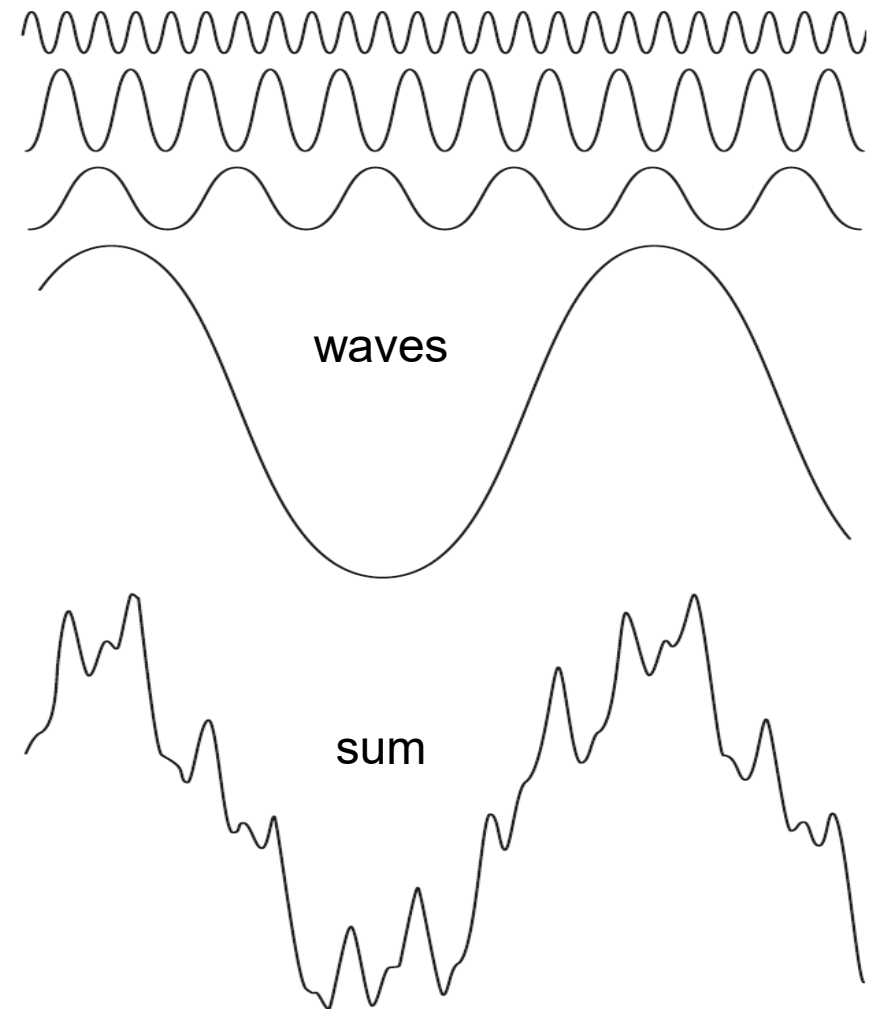
a_i is the weight (amplitude)

ω_i is the radial frequency

φ_i is the phase

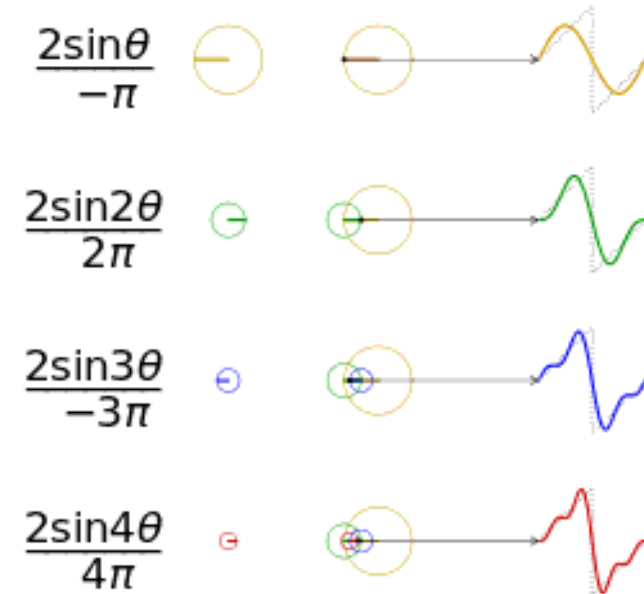
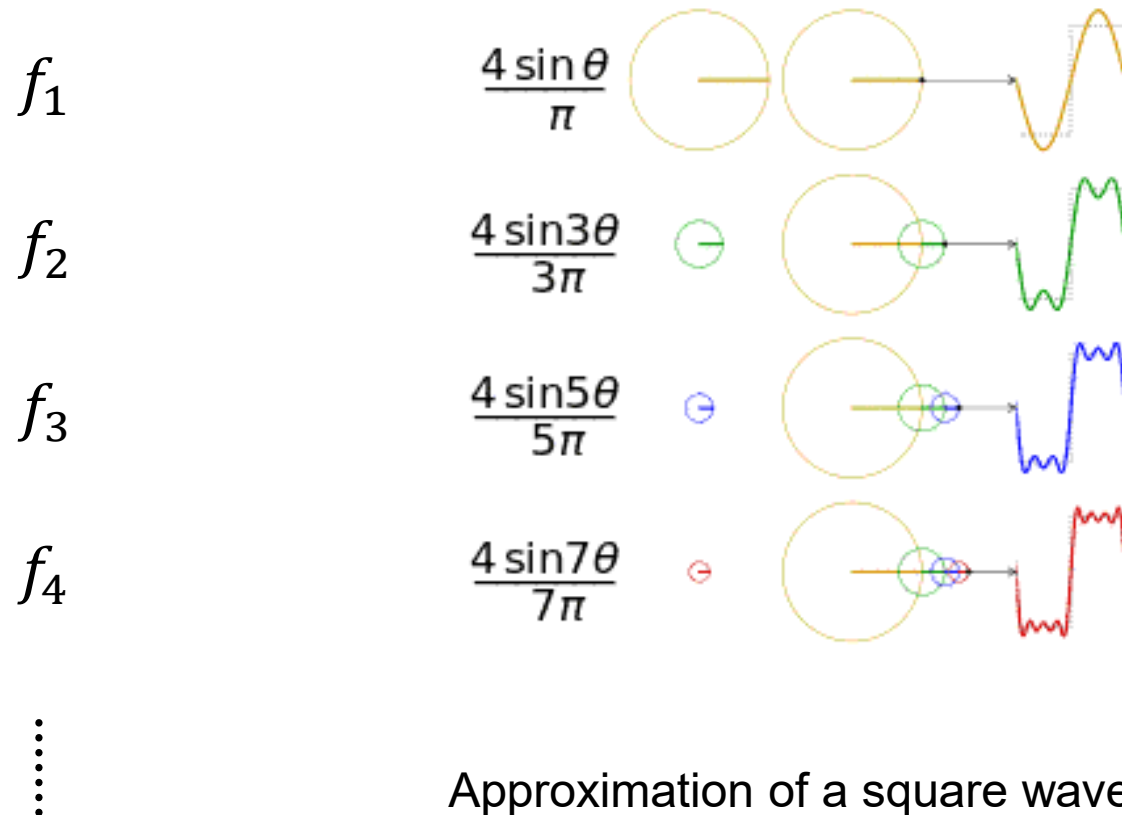
Add enough of them and you can get

any signal you want: $f = f_0 + f_1 + f_2 + f_3 + \dots$



Weighted sum of sines

https://en.wikipedia.org/wiki/Fourier_series



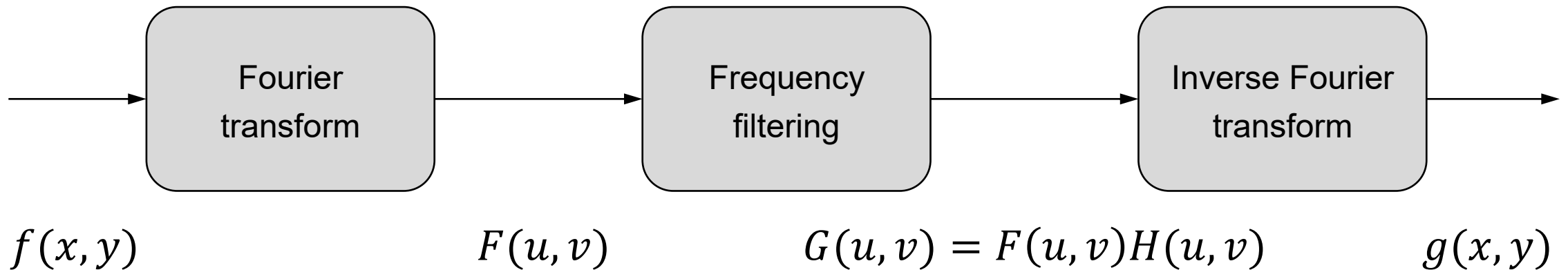
Approximation of a sawtooth wave

Spatial versus frequency domain

- **Spatial domain**
 - The image plane itself
 - Direct manipulation of pixels
 - Changes in pixel position correspond to changes in the scene
- **Frequency domain**
 - Fourier transform of an image
 - Directly related to rate of changes in the image
 - Changes in pixel position correspond to changes in the frequency

Frequency domain overview

- High frequencies correspond to rapidly changing intensities across pixel (e.g., edges, noise)
- Low frequency components correspond to large-scale image structures
- Frequency domain image processing via the Fourier transform



Definition of the Fourier transform (1D)

- **Forward Fourier transform**

$$F(u) = \int_{-\infty}^{\infty} f(x) e^{-i2\pi ux} dx$$

- **Inverse Fourier transform**

$$f(x) = \int_{-\infty}^{\infty} F(u) e^{i2\pi ux} du$$

Uses complex valued sinusoids

Notation

$f(x)$ is the spatial input function

$F(u)$ is the Fourier transform

$$e^{i\omega x} = \cos(\omega x) + i \sin(\omega x)$$

$\omega = 2\pi u$ is radial frequency

u is spatial frequency

$$i = \sqrt{-1}$$

Properties of the Fourier transform

Property	Spatial	Frequency
Superposition	$f_1(x) + f_2(x)$	$F_1(u) + F_2(u)$
Translation	$f(x - \Delta x)$	$F(u)e^{-i2\pi u\Delta x}$
Convolution	$f(x) * h(x)$	$F(u)H(u)$
Correlation	$f(x) \otimes h(x)$	$F(u)H^*(u)$
Multiplication	$f(x)h(x)$	$F(u) * H(u)$
Scaling	$f(ax)$	$F(u/a)/a$
Differentiation	$f^{(n)}(x)$	$(i2\pi u)^n F(u)$

Definition of the Fourier transform (2D)

- **Forward Fourier transform**

$$F(u, v) = \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} f(x, y) e^{-i2\pi(ux + vy)} dx dy$$

- **Inverse Fourier transform**

$$f(x, y) = \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} F(u, v) e^{i2\pi(ux + vy)} du dv$$

Uses complex valued sinusoids

Fourier transform pair

$$f \leftrightarrow F$$

$$F = R + i I$$

Real part + Imaginary part

Magnitude

$$|F| = \sqrt{R^2 + I^2}$$

Phase

$$\Phi = \arctan(I/R)$$

Discrete Fourier transform (DFT)

Digital images are discrete 2D functions

- **Forward discrete Fourier transform**

$$F(u, v) = \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} f(x, y) e^{-i2\pi\left(\frac{ux}{M} + \frac{vy}{N}\right)}$$

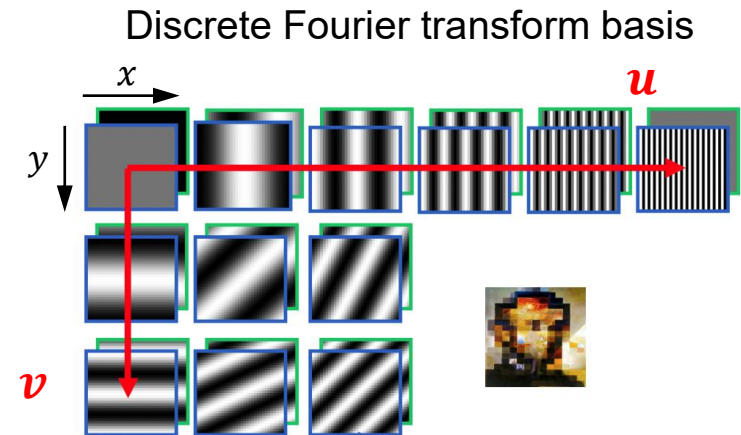
for $u = 0 \dots M - 1$ and $v = 0 \dots N - 1$

- **Inverse discrete Fourier transform**

$$f(x, y) = \frac{1}{MN} \sum_{u=0}^{M-1} \sum_{v=0}^{N-1} F(u, v) e^{i2\pi\left(\frac{ux}{M} + \frac{vy}{N}\right)}$$

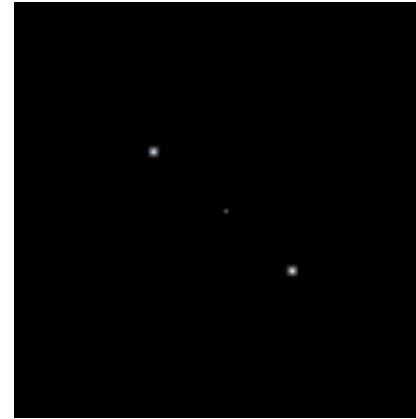
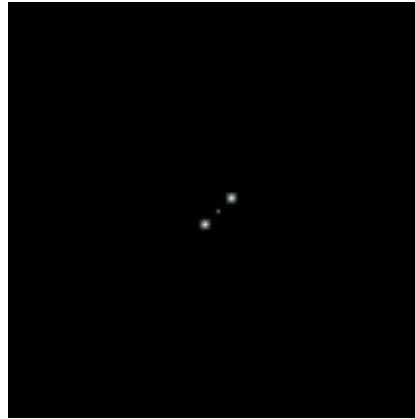
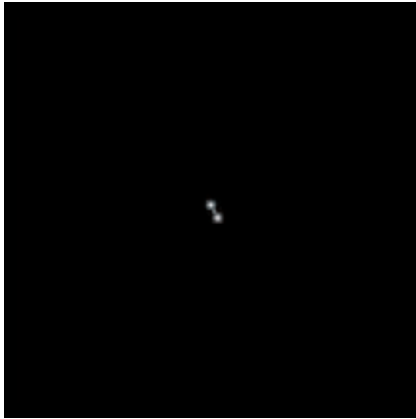
for $x = 0 \dots M - 1$ and $y = 0 \dots N - 1$

The discrete Fourier transform and its inverse always exist

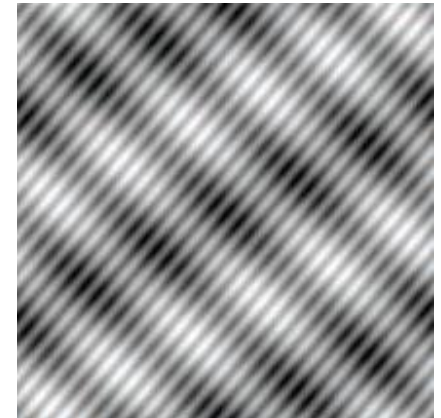
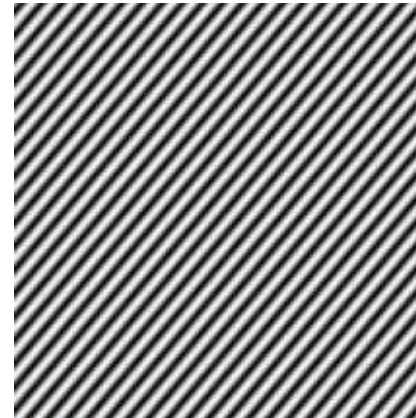
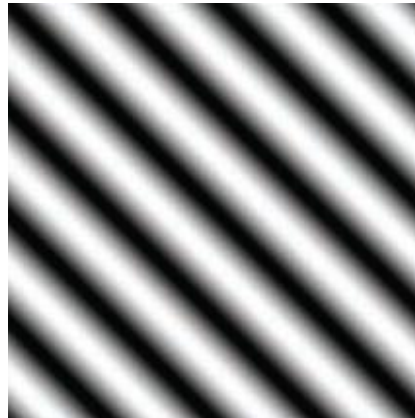


Examples of DFT basis images

$|F(u, v)|$



$f(x, y)$

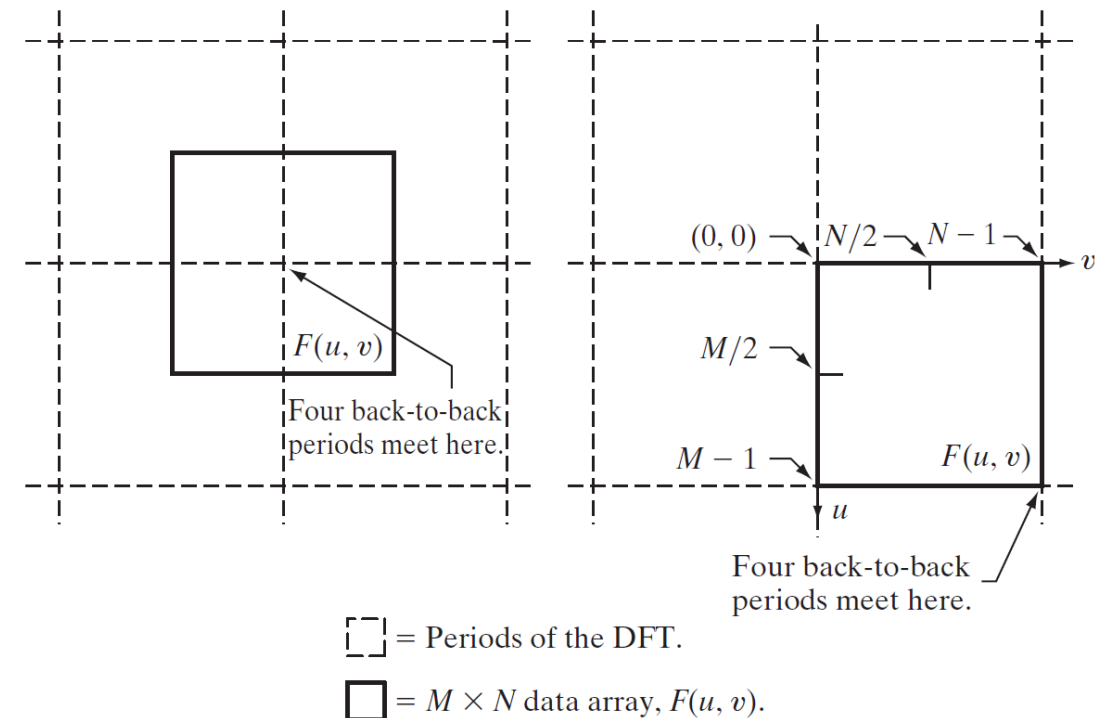
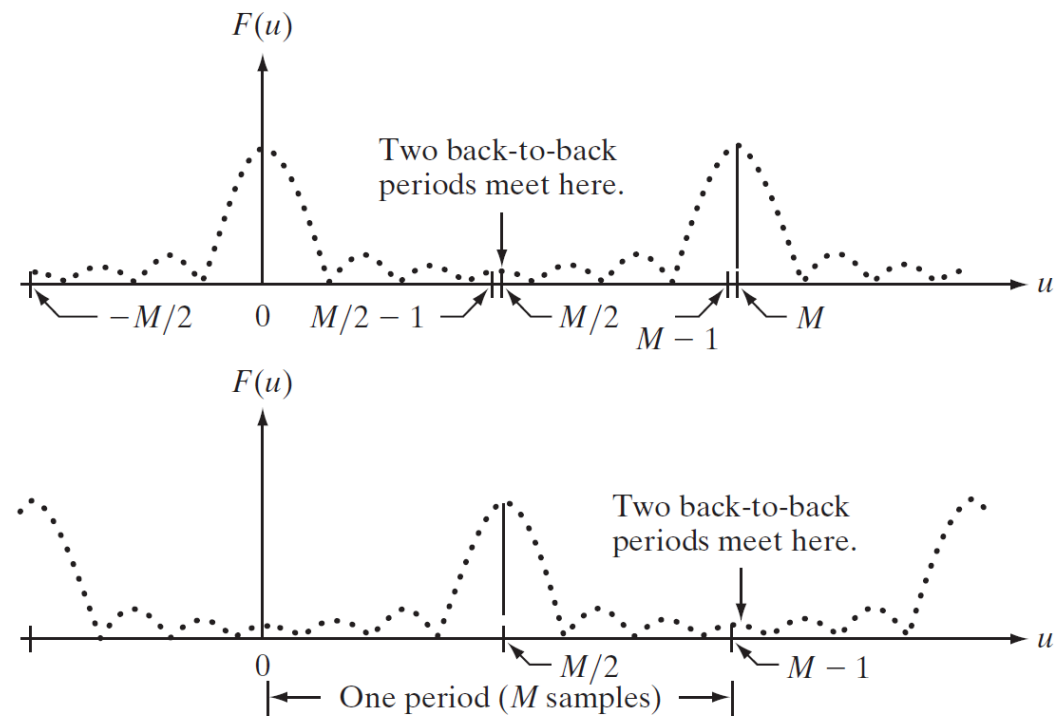


Frequency domain filtering

- Each $F(u, v)$ depends on all $f(x, y)$, $x = 0 \dots M - 1$, $y = 0 \dots N - 1$
- Frequencies in the Fourier domain correspond to patterns in the image
- The value of $F(0,0)$ is the total intensity over all pixels of the image
- Low frequencies correspond to slowly varying intensities across pixels
- High frequencies correspond to rapid intensity changes across pixels
- Noise typically corresponds to fluctuations in the highest frequencies

Frequency domain filtering

- Fourier images are typically centred for visualisation and processing



Frequency domain filtering

$$F(u, v) = \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} f(x, y) e^{-i2\pi\left(\frac{ux}{M} + \frac{vy}{N}\right)}$$

- Centering the Fourier images means multiplying the spatial images by $(-1)^{x+y}$

Translation property: $F(u - u_0, v - v_0) \leftrightarrow f(x, y) e^{i2\pi\left(\frac{u_0x}{M} + \frac{v_0y}{N}\right)}$

Centering by translation over: $u_0 = M/2$ and $v_0 = N/2$

Substitution yields: $F(u - M/2, v - N/2) \leftrightarrow f(x, y) e^{i\pi(x+y)}$

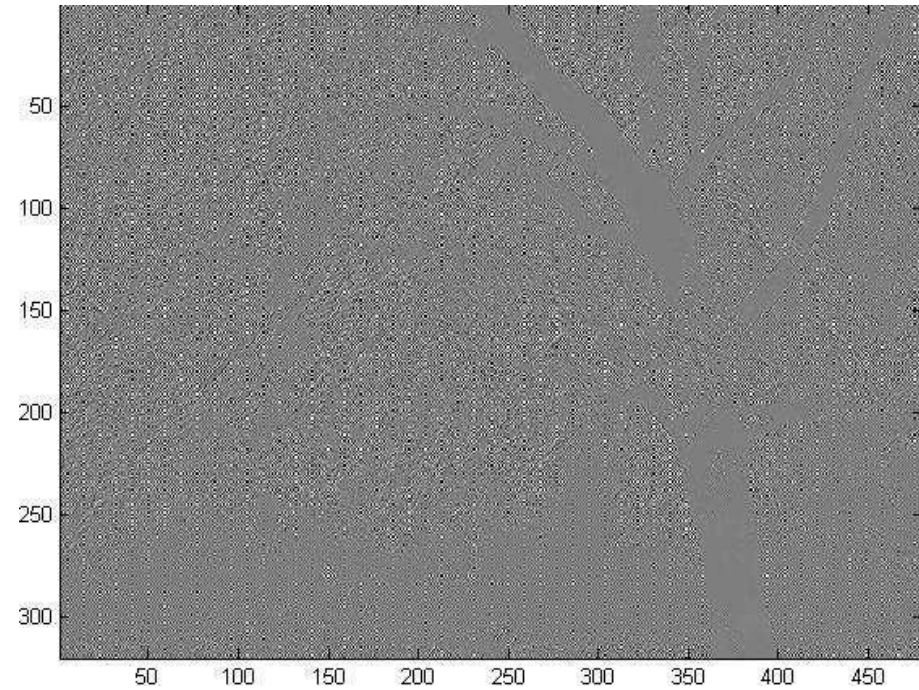
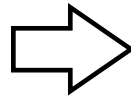
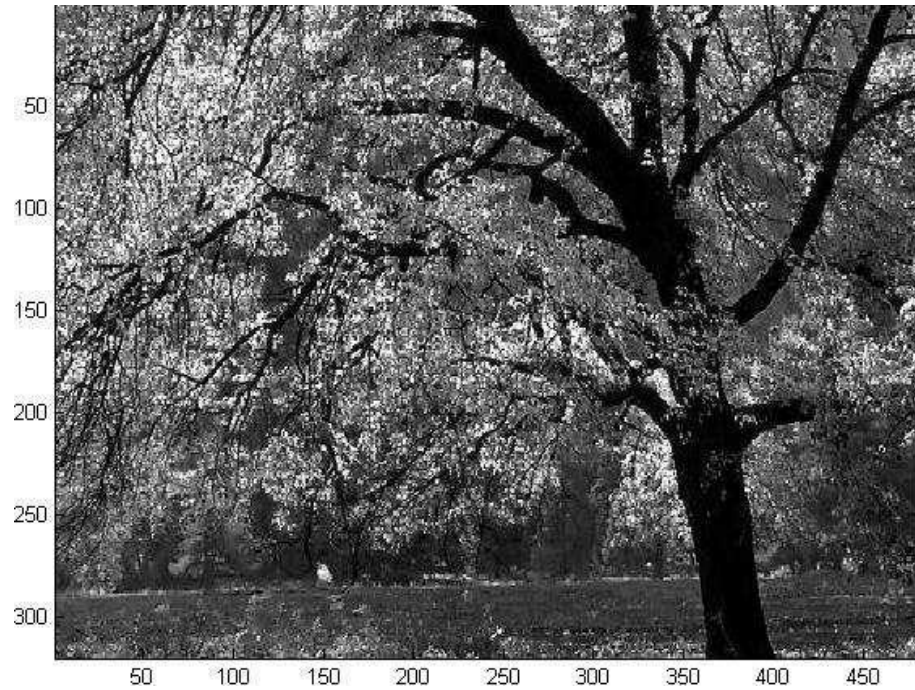
Which boils down to: $f(x, y) \left(\underset{\substack{\uparrow \\ = -1 \text{ or } 1}}{\cos(\pi(x+y))} + i \underset{\substack{\uparrow \\ = 0}}{\sin(\pi(x+y))} \right) = f(x, y) (-1)^{x+y}$ $x+y$ has integer value

Procedure for frequency domain filtering

1. Multiply the input image $f(x, y)$ by $(-1)^{x+y}$ to ensure centering $F(u, v)$
2. Compute the transform $F(u, v)$ from image $f(x, y)$ using the 2D DFT
3. Multiply $F(u, v)$ by a centred filter $H(u, v)$ to obtain the result $G(u, v)$
4. Compute the inverse 2D DFT of $G(u, v)$ to obtain the spatial result $g(x, y)$
5. Take the real component of $g(x, y)$ (the imaginary component is zero)
6. Multiply the result by $(-1)^{x+y}$ to remove the pattern introduced in step 1

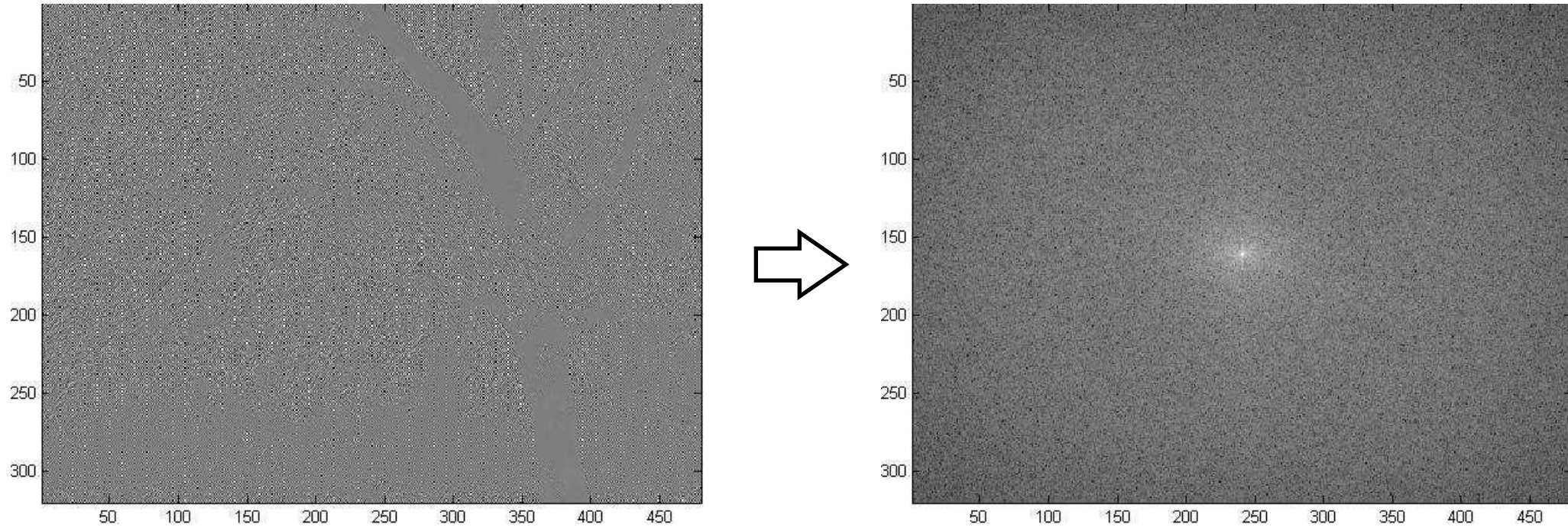
Example: low-pass filtering

1. Multiply the input image $f(x, y)$ by $(-1)^{x+y}$ to ensure centering $F(u, v)$



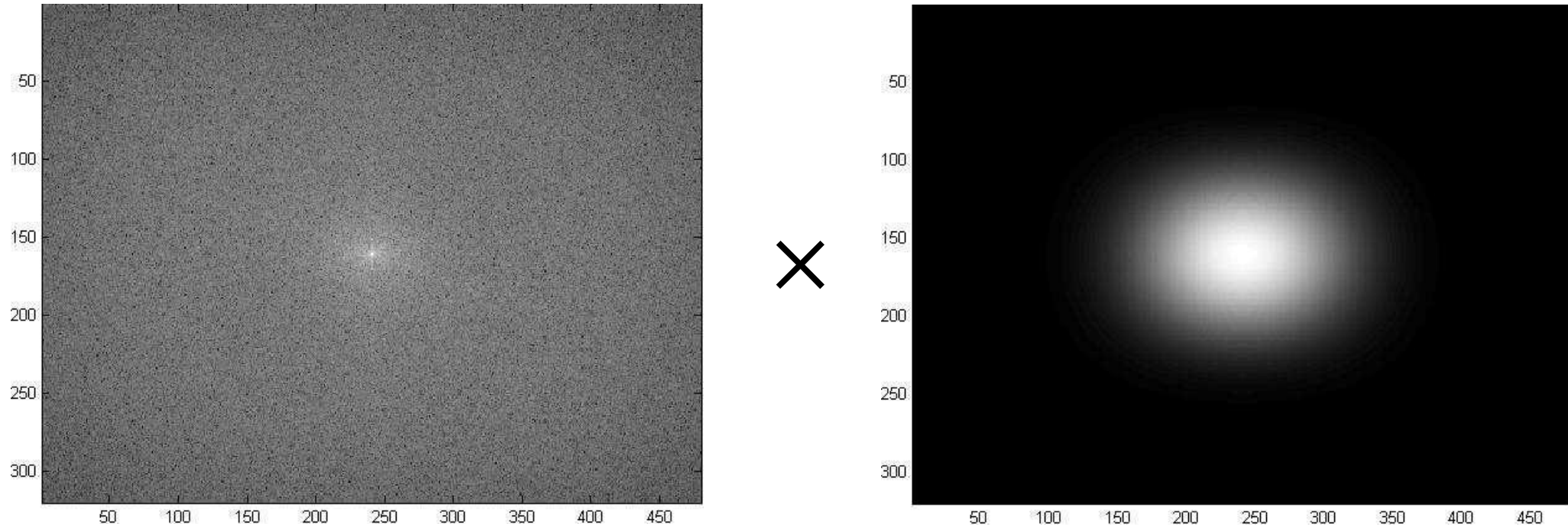
Example: low-pass filtering

2. Compute the transform $F(u, v)$ from image $f(x, y)$ using the 2D DFT



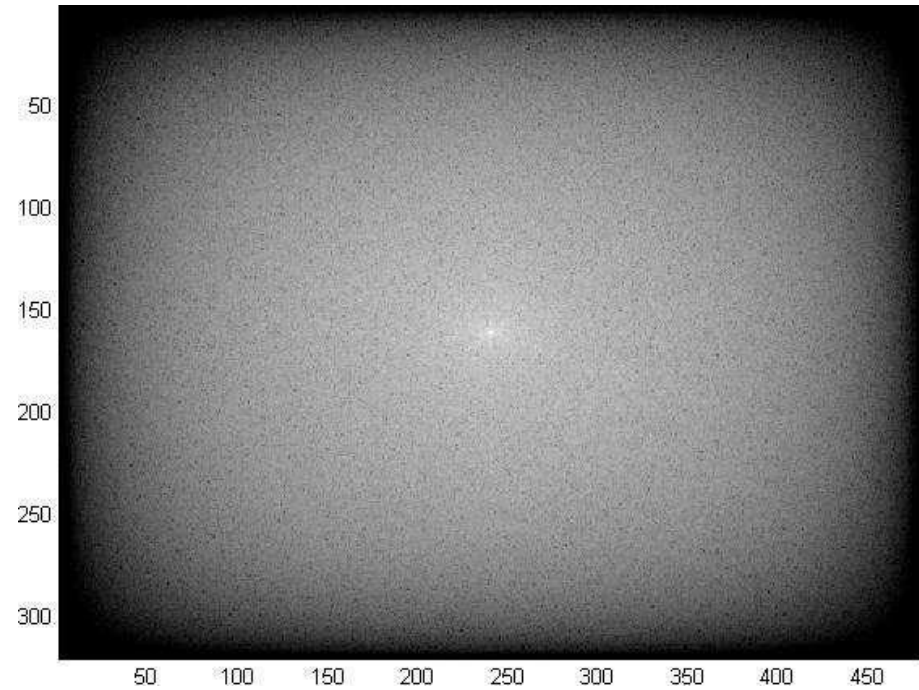
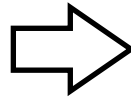
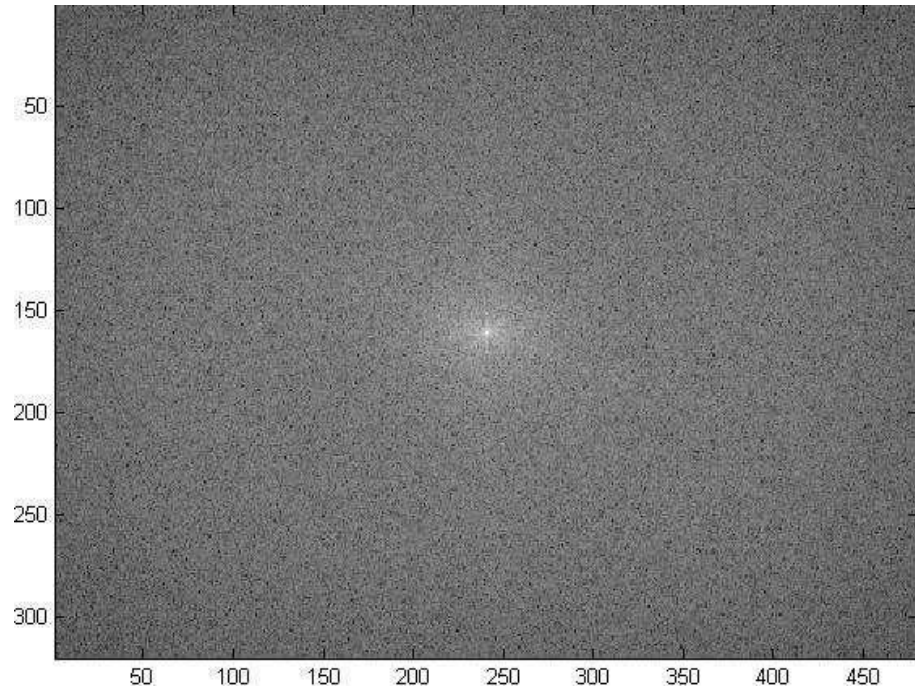
Example: low-pass filtering

3. Multiply $F(u, v)$ by a centred filter $H(u, v)$ to obtain the result $G(u, v)$



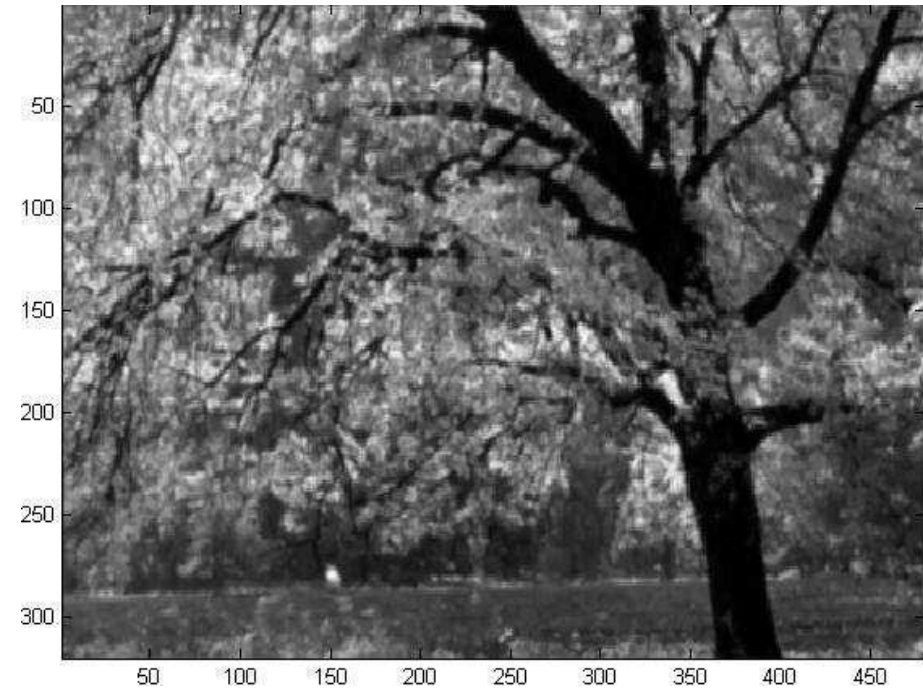
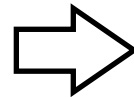
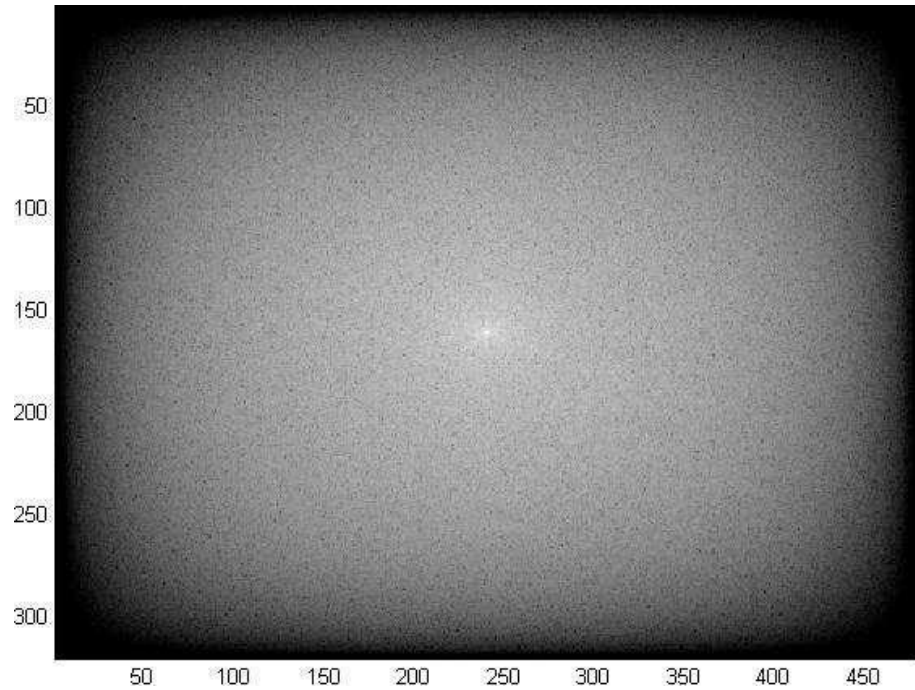
Example: low-pass filtering

3. Multiply $F(u, v)$ by a centred filter $H(u, v)$ to obtain the result $G(u, v)$



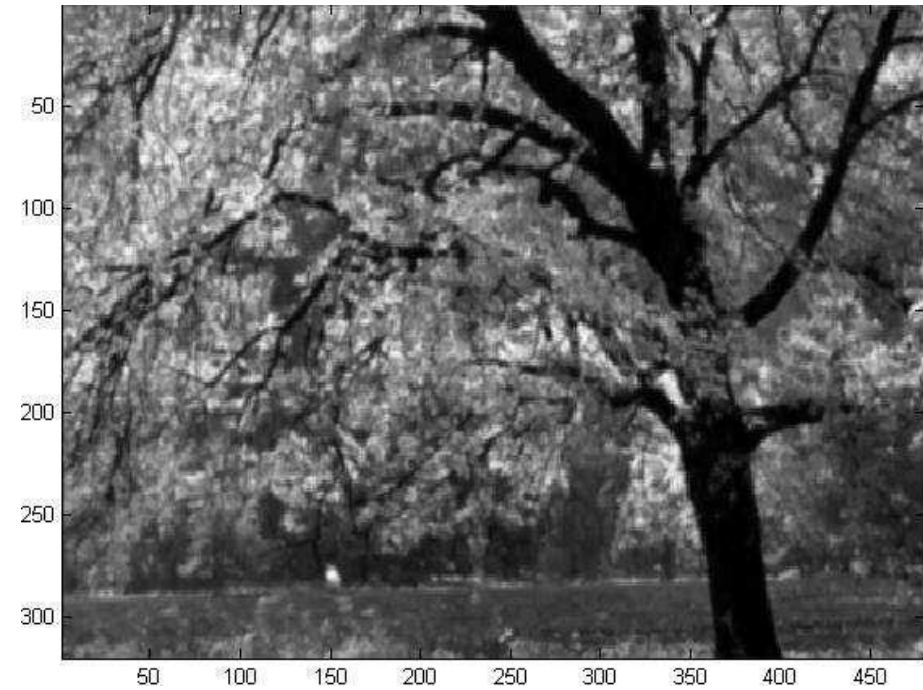
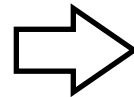
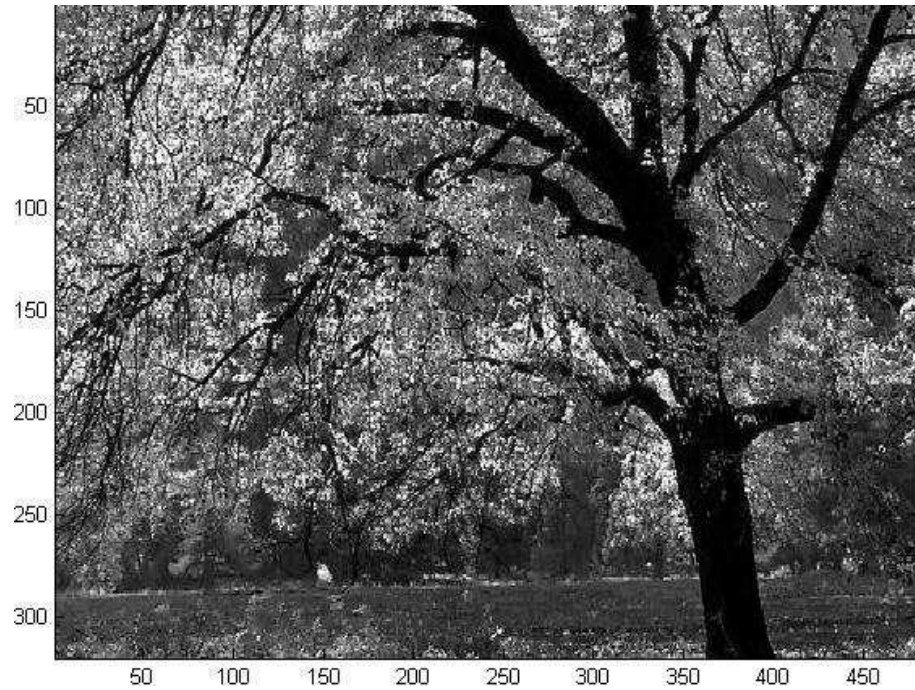
Example: low-pass filtering

4. Compute inverse DFT 5. Take real component 6. Multiply by $(-1)^{x+y}$



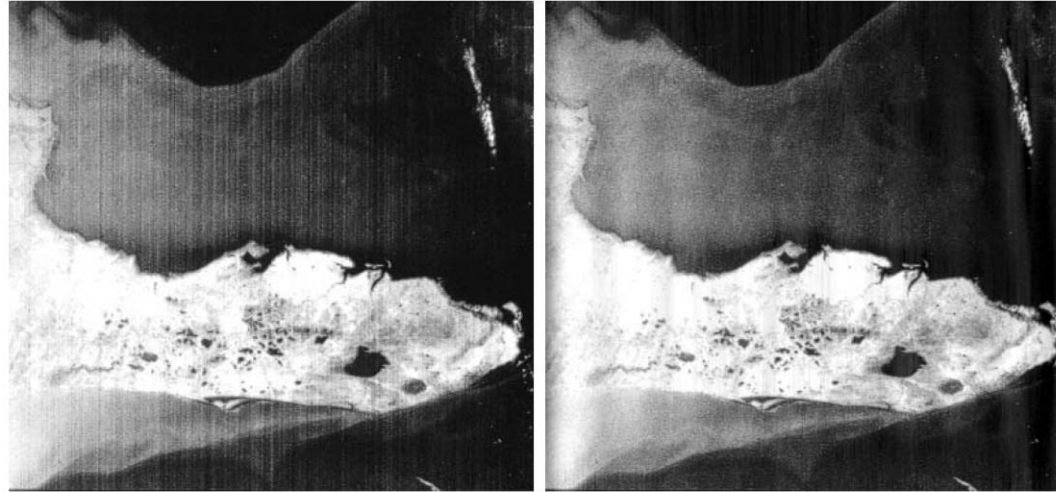
Example: low-pass filtering

- The resulting low-pass filtered image is a blurred version of the original



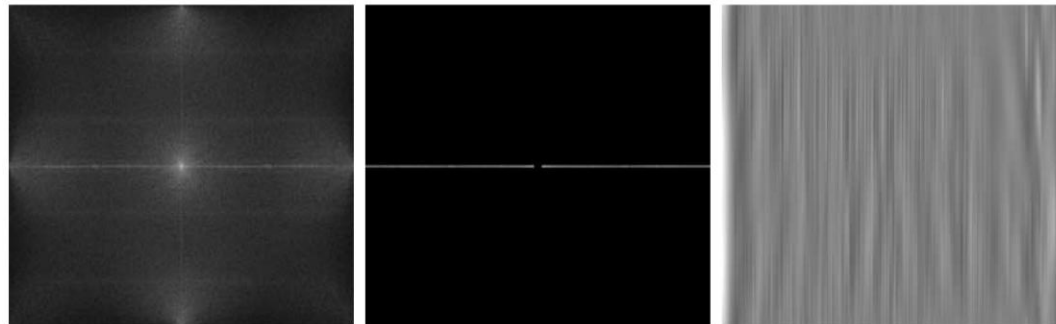
Example: notch filtering

Satellite image showing
scanline artifacts



Result image using the
notch reject filter

Fourier transform of
the satellite image



Noise pattern captured
by the **notch pass filter**

Notch pass filter

Exploiting the convolution theorem

- Filtering in the frequency domain can be computationally more efficient
- Designing filters can be more intuitive in the frequency domain
- Low-pass filter: keep low frequencies but attenuate high frequencies
- High-pass filter: keep high frequencies but attenuate low frequencies
- Band-pass filter: keep frequencies in a given band and attenuate the rest
- Take the inverse transform to get the corresponding spatial filter

Gaussian filter (low-pass)

- The Fourier transform of a Gaussian is a Gaussian

$$\text{1D:} \quad h(x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{x^2}{2\sigma^2}} \quad \leftrightarrow \quad H(u) = e^{-2\pi^2\sigma^2 u^2}$$

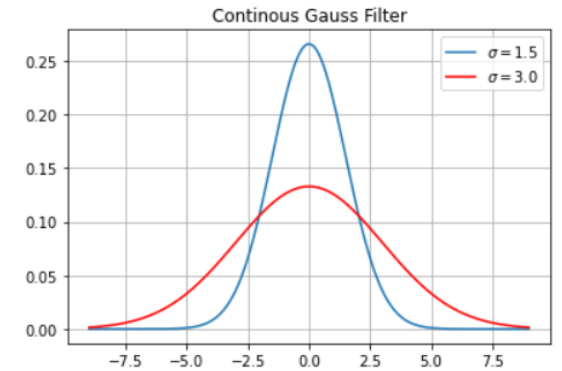
$$\text{2D:} \quad h(x, y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2 + y^2}{2\sigma^2}} \quad \leftrightarrow \quad H(u, v) = e^{-2\pi^2\sigma^2(u^2 + v^2)}$$

$$\text{nD:} \quad h(x_1, \dots, x_n) = \left(\frac{1}{2\pi\sigma^2}\right)^{n/2} e^{-\frac{x_1^2 + \dots + x_n^2}{2\sigma^2}} \quad \leftrightarrow \quad H(u_1, \dots, u_n) = e^{-2\pi^2\sigma^2(u_1^2 + \dots + u_n^2)}$$

Gaussian filter

Many nice properties motivate the use of the Gaussian filter:

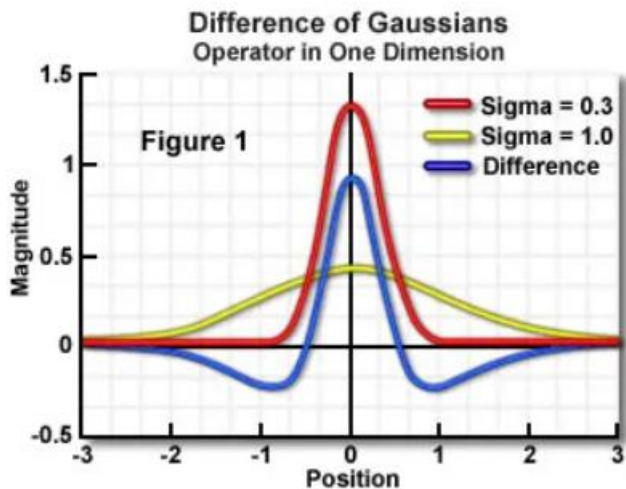
- It is the only filter that is both separable and circularly symmetric
- The Fourier transform of a Gaussian is also a Gaussian function
- It has optimal joint localization in spatial and frequency domains
- The n -fold convolution of any low-pass filter converges to a Gaussian
- It is infinitely smooth so it can be differentiated to any desired degree
- It scales naturally (sigma) and allows for consistent scale-space theory



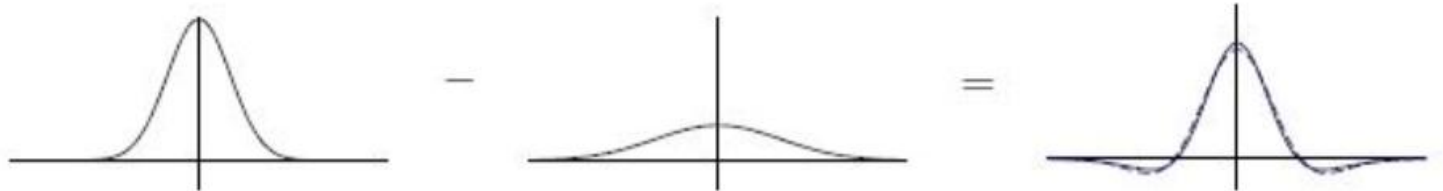
Difference of Gaussian (DoG)

- DoG filter: useful for edge detection while removing some noise, depending on the filter design

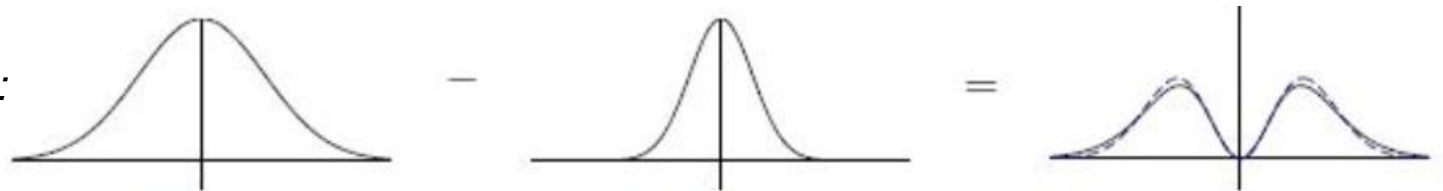
$$\text{DoG}(x) = a_1 h(x; \sigma_1) - a_2 h(x; \sigma_2) \quad \leftrightarrow \quad \text{DoG}(u, v) = a_1 H(u; \sigma_1) - a_2 H(u; \sigma_2)$$



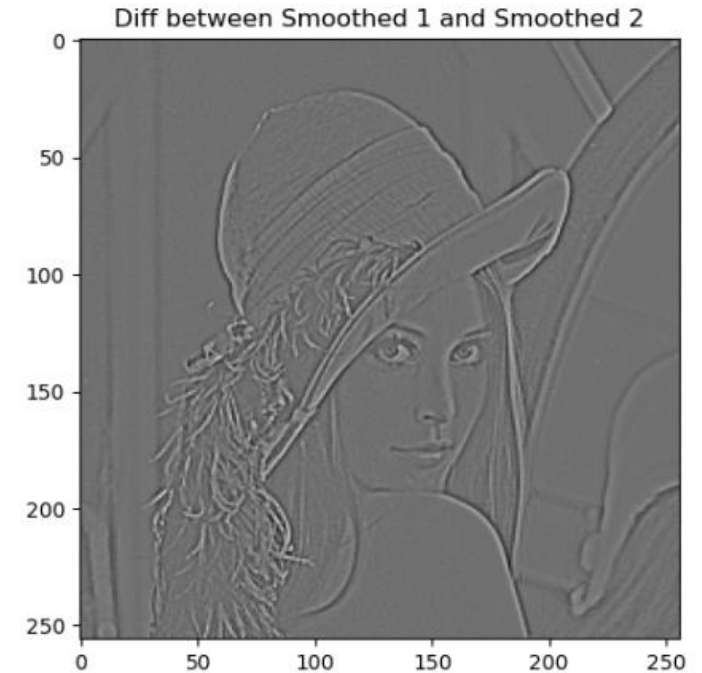
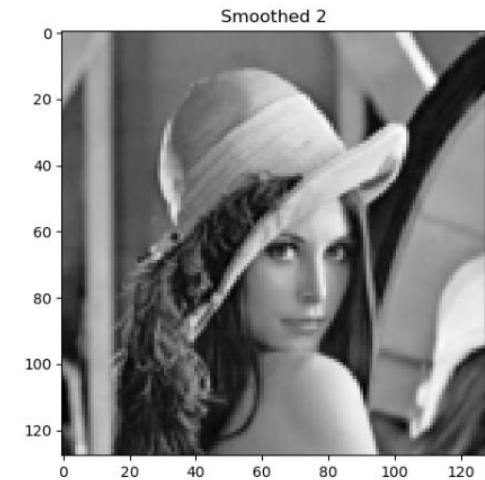
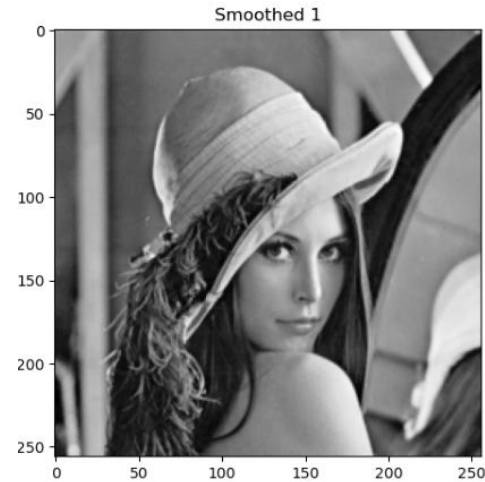
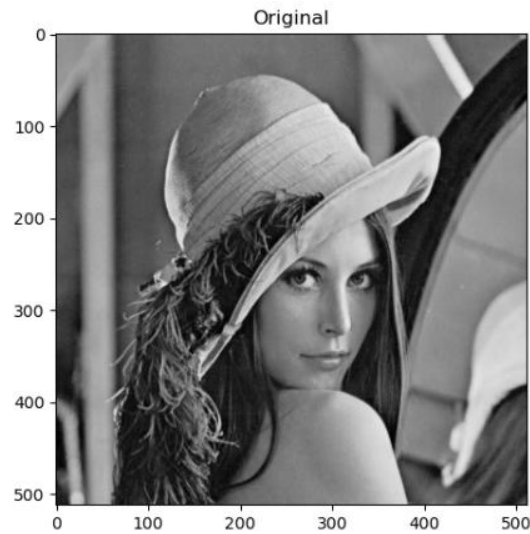
Spatial:



Frequency:

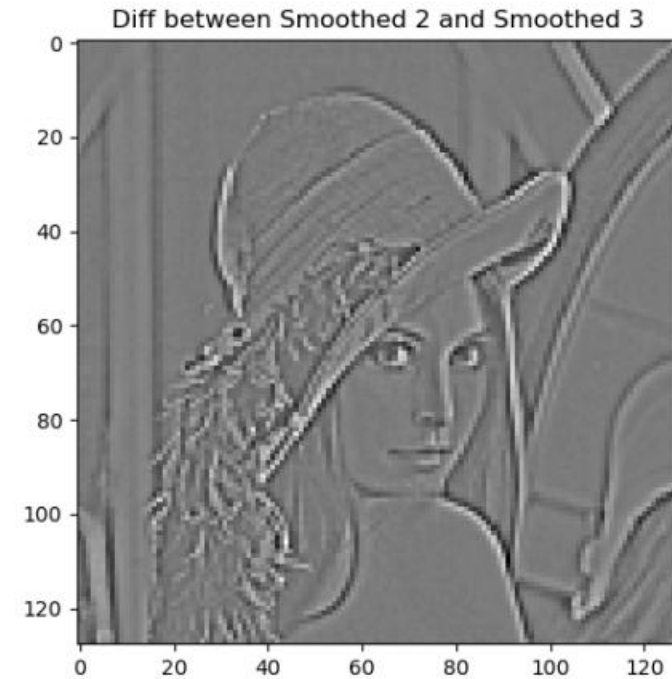
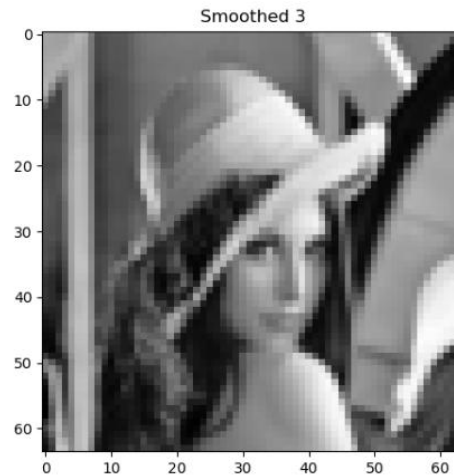
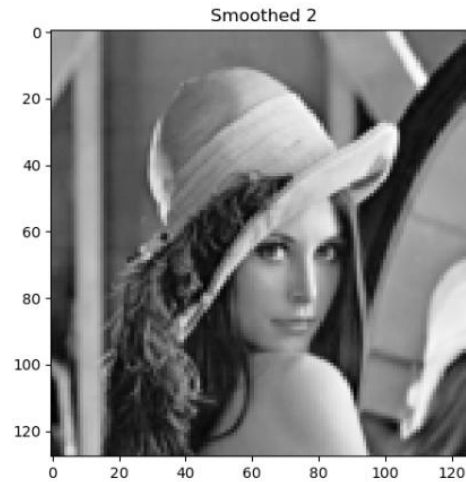
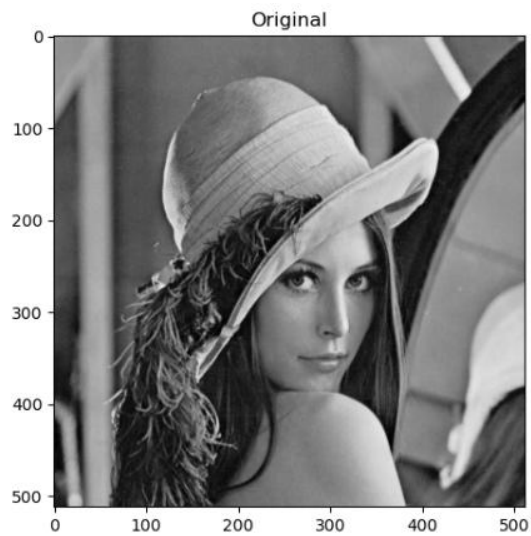


Difference of Gaussian (DoG)



<https://maucher.pages.mi.hdm-stuttgart.de/orbook/preprocessing/05GaussianLowPassFilter.html>

Difference of Gaussian (DoG)



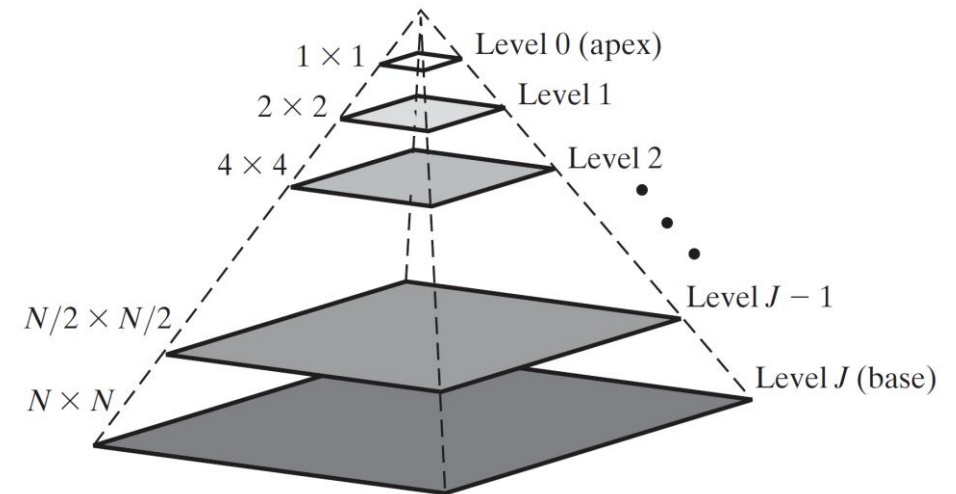
<https://maucher.pages.mi.hdm-stuttgart.de/orbook/preprocessing/05GaussianLowPassFilter.html>

What is lost when lowering resolution?

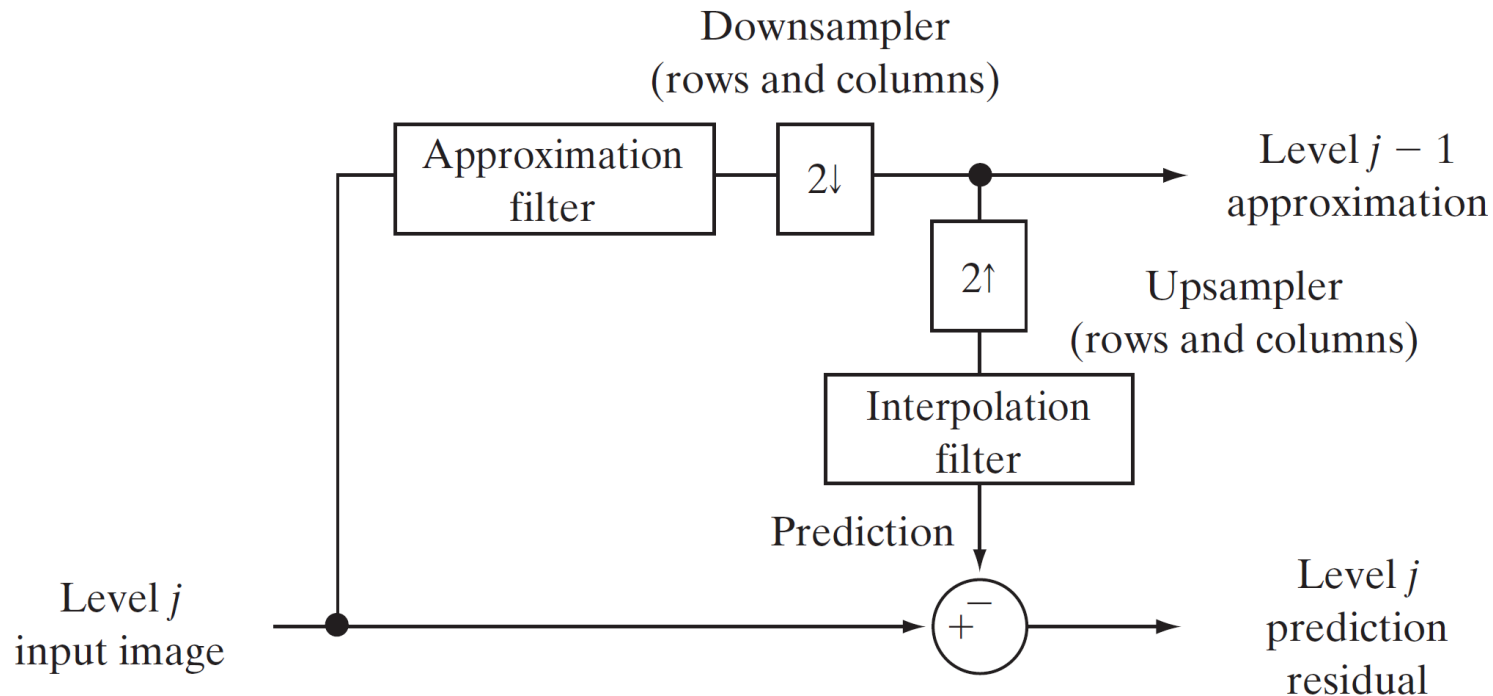


Multiresolution image processing

- Small objects and fine details benefit from high resolution
- Large objects and coarse structures can make do with lower resolution
- If both are present at the same time, multiple resolutions may be useful
- This requires computing image pyramids



Creating image pyramids

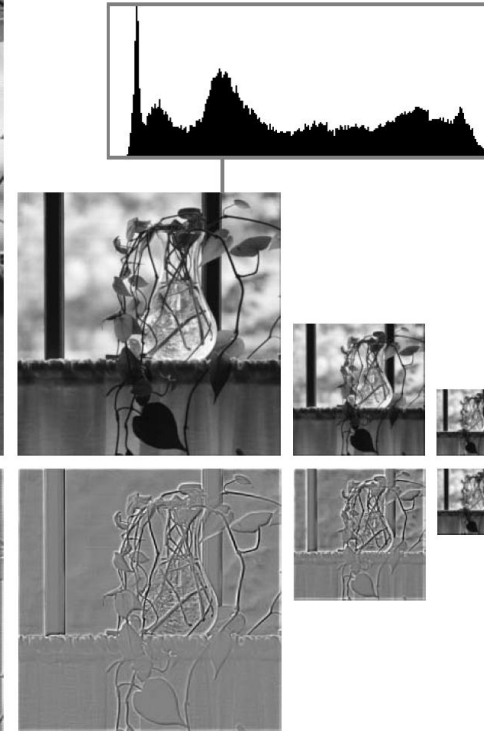
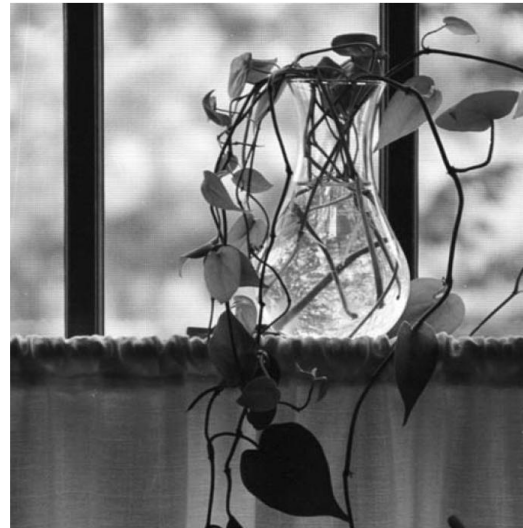


Repeating this produces an approximation and prediction residual pyramid

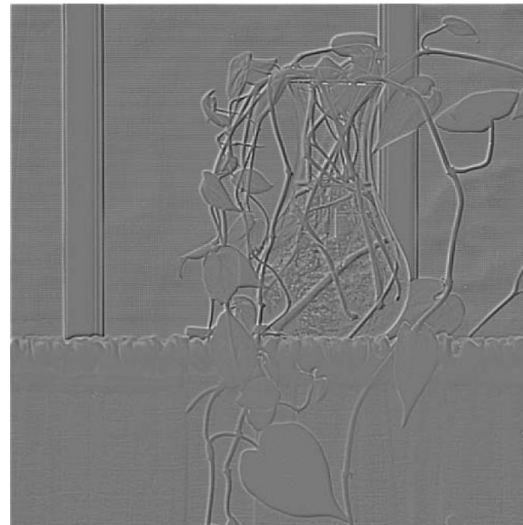
1. Compute an approximation of the input image by filtering and downsampling
2. Upsample the output of step 1 and filter the result (interpolation)
3. Compute the difference between the prediction of step 2 and the input to step 1

Example

Approximation pyramid



Residual pyramid

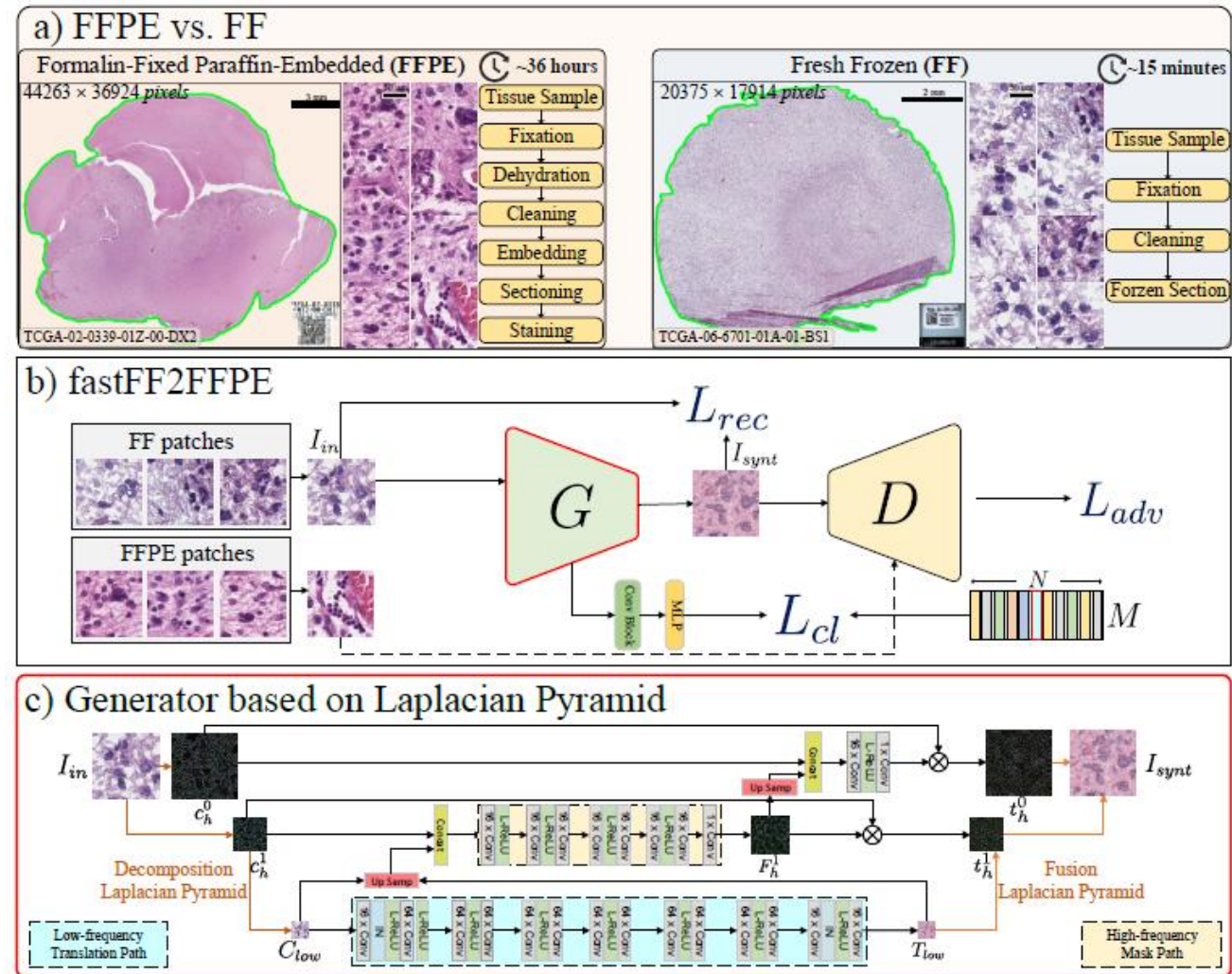


To reconstruct the image:

1. Upsample and filter the lowest resolution approximation image
2. Add the one-level higher prediction residual

Example

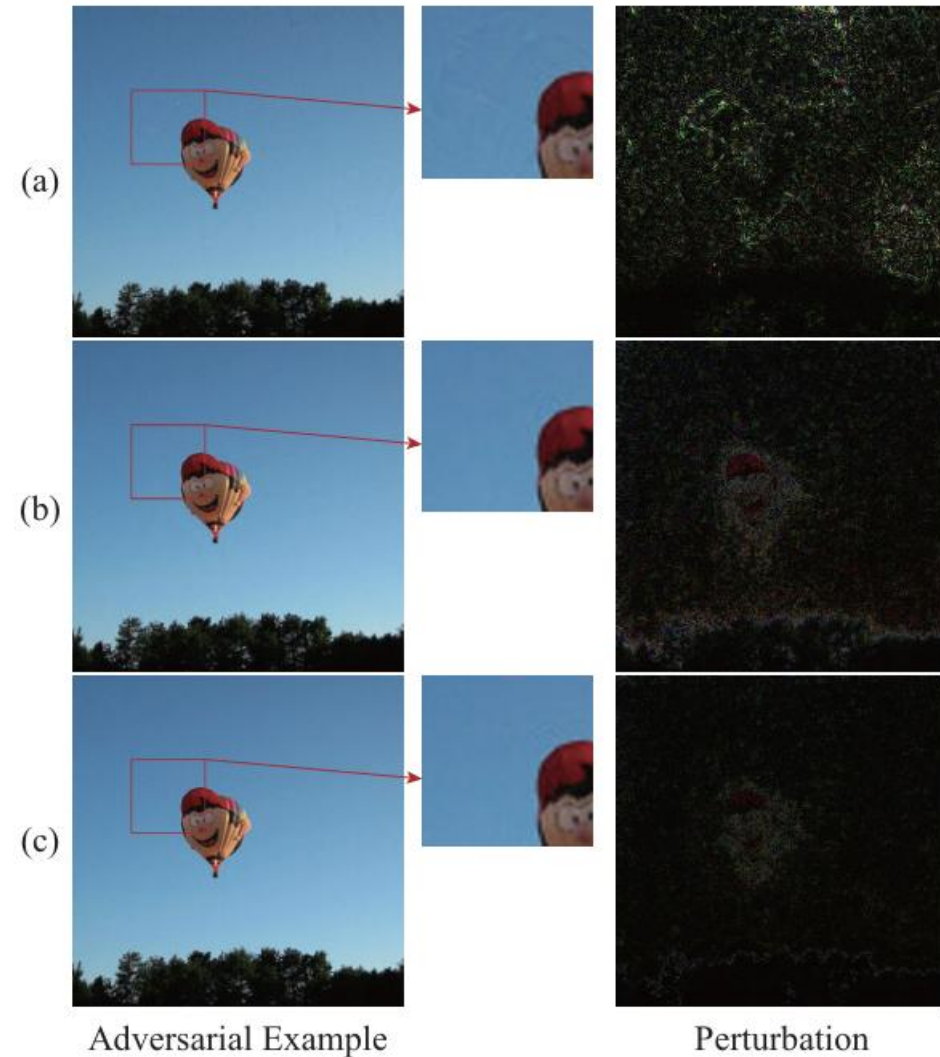
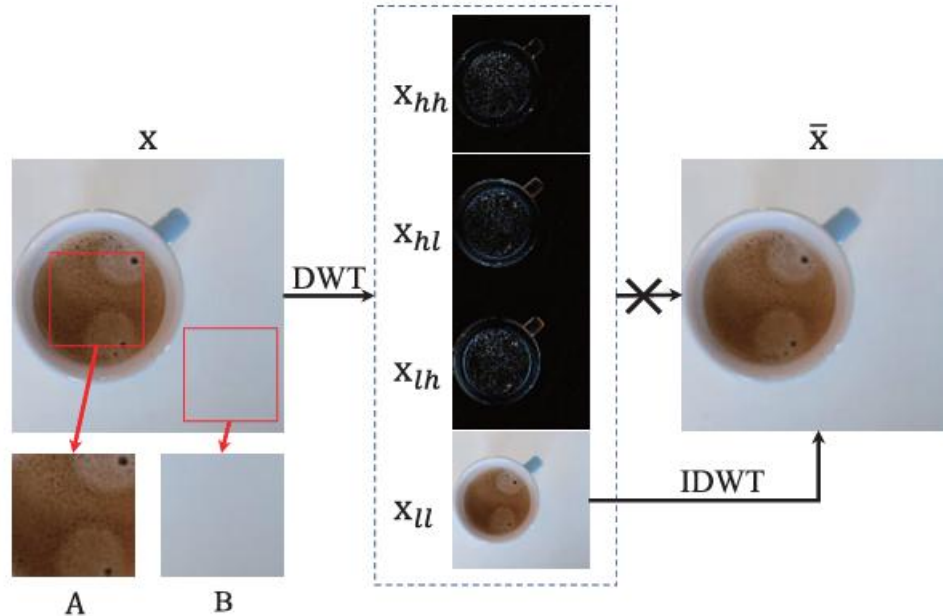
Integration of Laplacian Pyramid with deep learning for image style transfer



L. Fan et al., Fast FF-to-FFPE Whole Slide Image Translation via Laplacian Pyramid and Contrastive Learning, MICCAI 2022

Example

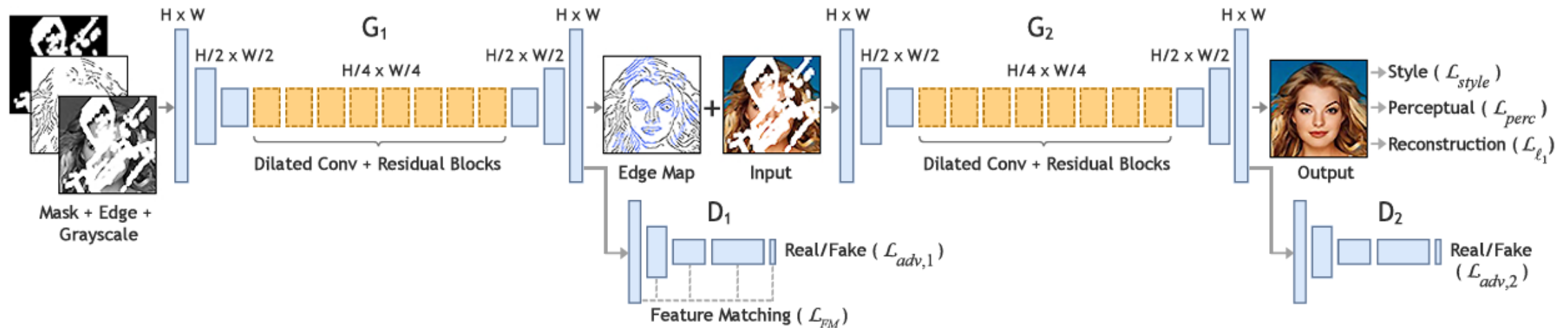
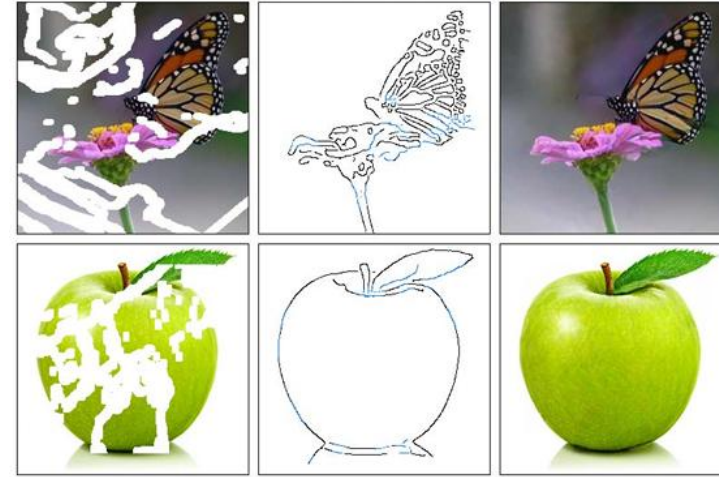
Adversarial perturbation on high-frequency components



C. Luo et al., Frequency-driven Imperceptible Adversarial Attack on Semantic Similarity, CVPR 2022

Example

Utilising edge information for image generation



K. Nazeri et al., EdgeConnect: Structure Guided Image Inpainting using Edge Prediction, ICCVW 2019

Further reading on discussed topics

- Chapter 4 of Gonzalez and Woods 2002
- Sections 3.4-3.5 of Szeliski 2010

Acknowledgement

- Some images drawn from the above resources

Example exam question

Which one of the following statements on image filtering is incorrect?

- A. Median filtering reduces noise in images.
- B. Low-pass filtering results in blurry images.
- C. High-pass filtering smooths fine details in images.
- D. Notch filtering removes specific image frequencies.